

CryptoLens

Phase 2 Deliverables Document

Bronze & Silver Lakehouse Implementation

Field	Detail
Team Members	Abdullah Saeed (24L-2529); Arqum Umais (24L-2591)
GitHub Repository	https://github.com/x-abdullah-saeed-x/CryptoLens
Platform	Databricks Free Edition (Serverless) — Apache Spark, Delta Lake, Unity Catalog
Document Scope	Phase 2: Bronze + Silver pipeline, schema enforcement, idempotency, audit logging

1. Overview

Phase 2 transitions CryptoLens from a one-off download script into an enterprise-pattern PySpark pipeline. This document maps each Phase 2 requirement to its concrete implementation in `01_bronze_ingest.ipynb` and `02_silver_transform.ipynb`, both committed to the GitHub repository.

2. Infrastructure & Environment

- Workspace: Databricks Free Edition (Serverless SQL/Compute), Unity Catalog enabled.
- Storage: raw files land in a Unity Catalog Volume (`workspace.cryptolens.bronze_raw`); all structured tables (Bronze, Silver, `dim_symbol`, audit logs) are managed Delta tables in the `workspace.cryptolens` schema.
- Version control: both notebooks, this document, and the data dictionary are committed to the GitHub repository; Databricks Git folders keep the workspace copy in sync via `commit/push`.
- Environment constraint: Free Edition's serverless compute restricts outbound internet access to an internal allowlist that excludes Binance's domains (confirmed via direct DNS resolution testing — `api.binance.com` and `data.binance.vision` fail to resolve, while `pypi.org` and `github.com` succeed). Raw file acquisition therefore runs on a local machine and verified files are pushed into the Volume via the Databricks CLI (`databricks fs cp`).

3. Data Modeling & Schema Enforcement

Full column-level definitions are provided in the separate Data Dictionary (`CryptoLens_Data_Dictionary.pdf`). Summary of how each requirement is met:

Requirement	Implementation
No inferSchema	RAW_KLINE_SCHEMA and LOG_SCHEMA are explicit StructType/StructField definitions in both notebooks; every <code>.csv()</code> read passes <code>.schema(...)</code> explicitly.
Strict data types & casting	Bronze keeps Binance's native LONG/DOUBLE types. Silver casts <code>open_time/close_time</code> (epoch ms) to <code>TIMESTAMP</code> , OHLCV fields to <code>DECIMAL(18,8)</code> for financial precision, and <code>number_of_trades</code> to <code>INTEGER</code> .
load_timestamp on every record	Added via <code>current_timestamp()</code> on every write, in both <code>bronze_klines/bronze_exchange_info</code> and <code>silver_klines/dim_symbol</code> .

4. Pipeline Robustness & Idempotency

4.1 Idempotent execution (MERGE INTO)

- Bronze: MERGE on (`symbol`, `interval`, `load_type`, `open_time`) with `WHEN NOT MATCHED THEN INSERT` — re-ingesting the same file inserts zero duplicate rows.
- Silver: MERGE on (`symbol`, `interval`, `open_time`) with `WHEN MATCHED THEN UPDATE / WHEN NOT MATCHED THEN INSERT` — also correctly handles a later full-load file overwriting a value from an earlier daily incremental file for the same timestamp.

- `dim_symbol`: MERGE with three branches — WHEN MATCHED (status/filter changed) THEN UPDATE, WHEN NOT MATCHED BY TARGET THEN INSERT (new listing), WHEN NOT MATCHED BY SOURCE THEN soft-delete (`is_active = false`, `status = 'DELISTED'`).

4.2 Parameterized backfills

Both notebooks expose Databricks widgets — `symbol`, `interval`, `load_type`, `year`, `month`, `day` — and build the Volume file path dynamically from these parameters (`build_volume_path()`). No function assumes "today"; any historical month/day can be re-processed by changing widget values and re-running.

4.3 Schema drift handling

- CSV reads use `mode='PERMISSIVE'` with `columnNameOfCorruptRecord='_corrupt_record'` — rows that don't match `RAW_KLINE_SCHEMA` populate this column instead of failing the batch.
- Rows with a populated `_corrupt_record` are written to `bronze_quarantine` (with option(`"mergeSchema","true"`) so the quarantine table itself can tolerate evolving bad-row shapes) rather than crashing the pipeline.
- Delta writes to Bronze use `mergeSchema=true` to gracefully evolve the table if Binance adds a column in the future.

5. Audit & Execution Logging

`pipeline_execution_logs` (Delta table, schema in the Data Dictionary) receives one row from every call to `log_execution()`, on every success, failure, and skip path in both notebooks. Logged fields: `layer` (Raw-to-Bronze / Bronze-to-Silver), the file path or `symbol-interval-loadtype` parameter processed, `load_type`, `start_time`, `end_time`, `status`, `rows_inserted`, `rows_updated`, and `error_message`.

This means every file processed across both Full and Incremental loads, at both layers, leaves an auditable trace — queryable directly as a Delta table for monitoring or troubleshooting.

6. Verification Performed

- Re-ran `01_bronze_ingest` and `02_silver_transform` with identical widget parameters twice in a row; row counts in `bronze_klines` and `silver_klines` were unchanged on the second run, confirming idempotency.
- Verified DNS-level network restriction on Free Edition serverless compute via a `direct socket.gethostbyname()` test against Binance vs. control domains, documented in the README.
- Confirmed `pipeline_execution_logs` captures a `SUCCESS` row per processed file and a `SKIPPED` row for any requested file not present in the Volume.